UCK 439E Term Project
Optimal B737–800 Flight Planning under a Wind Field

Yunus Emre Kara
Istanbul Technical University

June 2026

Abstract

We solve the cruise-phase trajectory-optimisation problem for a Boeing 737–800 under a polynomial wind field, using three different numerical optimal-control techniques: (i) direct transcription with Hermite–Simpson collocation and IPOPT, (ii) reduced two-dimensional dynamic programming with a numba-jitted Bellman backup, and (iii) proximal policy optimisation in a custom Gymnasium environment. Three flight scenarios from the project specification are solved by each method, eight standard plots are produced per flight, and a single overlay figure per metric compares the three methods. The direct method serves as the benchmark; DP and RL provide a methodological comparison.

1 Introduction

This report documents the term project for UCK 439E (Introduction to Optimal Control). The task statement is a B737–800 flight-planning problem with a known wind field $W(\lambda, \varphi)$ given by a ninth-order polynomial in (longitude, latitude). The aircraft performance model is BADA-style with ISA-tropospheric atmosphere. Three flight scenarios are prescribed (Tables **??** and **??**); each scenario fixes the start state $(\lambda_0, \varphi_0, h_0, V_0, \psi_0, m_0)$ and the destination $(\lambda_f, \varphi_f, h_f)$.

Methods chosen. The project requires (a) one direct or indirect optimal-control method and (b) either dynamic programming *or* reinforcement learning. We implement *both* (b) options to compare three families of techniques side-by-side.

2 Problem Formulation

2.1 State, control, and dynamics

The state vector is $\mathbf{x} = (x_e,\, y_n,\, h,\, V,\, \psi,\, m)^\top$ in flat-Earth East–North–Up (ENU) coordinates anchored at the per-flight reference origin (λ_0, φ_0). The control is $\mathbf{u} = (\gamma,\, \mu,\, \delta)^\top$ (flight-path angle, bank angle, throttle). The PDF dynamics are

$$\begin{aligned} \dot{x}_e &= V\cos\psi\cos\gamma + W_x(\lambda,\varphi), \\ \dot{y}_n &= V\sin\psi\cos\gamma + W_y(\lambda,\varphi), \\ \dot{h} &= V\sin\gamma, \\ \dot{V} &= \frac{T-D}{m} - g_0\sin\gamma, \\ \dot{\psi} &= \frac{C_L\, S\, \rho V}{2m}\,\frac{\sin\mu}{\cos\gamma}, \\ \dot{m} &= -f(V,h,\delta). \end{aligned} \tag{1}$$

Heading-angle convention. Because the kinematic equations use $\dot{x} = V\cos\psi$ and $\dot{y} = V\sin\psi$ with $(x, y) = (\text{east}, \text{north})$, ψ is measured *counter-clockwise from east* (mathematics convention), not clockwise from north (aviation convention). Hence $\psi = 0 \Rightarrow$ heading east; $\psi = \pi/2 \Rightarrow$ heading north. Table 3 entries such as "$\psi_0 = 0\,\text{deg}$" therefore mean an initial heading due east. This choice is consistent across all three methods.

2.2 Aerodynamic and propulsion sub-models

Atmosphere $\rho(h) = \rho_0(1-2.2257\times10^{-5}h)^{4.2586}$. Lift coefficient is closed-form from the coordinated-turn balance $L\cos\mu = mg$, giving $C_L = 2mg/(\rho S V^2\cos\mu)$, with polar drag $C_D = C_{D0} + kC_L^2$ and $D = \frac{1}{2}\rho V^2 S C_D$. Maximum thrust uses the BADA polynomial in h_{ft}, $T_{\max}(h) = C_{T,\text{cr}}\,C_{T,c1}(1 - h_{\text{ft}}/C_{T,c2} + C_{T,c3}h_{\text{ft}}^2)$, and the available thrust is $T = \delta\,T_{\max}(h)$. Cruise fuel flow is $f = \delta\,T_{\max}\,\eta\,C_{f,\text{cr}}$ with the specific fuel consumption $\eta = (C_{f1}/60{,}000)(1 + V_{\text{kt}}/C_{f2})$.

2.3 Wind field

The wind field is given by Table 2 of the project as a 9-term bivariate polynomial in (λ, φ) in degrees,

$$W_{\{x,y\}}(\lambda,\varphi) = c_0 + c_1\lambda + c_2\varphi + c_3\lambda\varphi + c_4\lambda^2 + c_5\varphi^2 + c_6\lambda^2\varphi + c_7\lambda\varphi^2 + c_8\lambda^2\varphi^2.$$

Because we integrate dynamics in ENU, we convert $(x_e, y_n) \to (\lambda, \varphi)$ inside the right-hand side using the flat-Earth approximation $\lambda = \lambda_0 + (x_e/(R\cos\varphi_0))\cdot 180/\pi$, $\varphi = \varphi_0 + (y_n/R)\cdot 180/\pi$. Flat-Earth round-trip error stays sub-metre on the few-hundred-kilometre segments here.

2.4 Cost functional

The PDF cost is

$$J = \big(x(t_f) - x_d\big)^2 + \big(y(t_f) - y_d\big)^2 + \big(h(t_f) - h_d\big)^2 + \int_0^{t_f}(0.05 + f)\,dt. \tag{2}$$

The Mayer terminal squared-distance term enforces arrival; the running term penalises both fuel and elapsed time (the constant 0.05 has units [kg/s] and biases against needlessly long trajectories).

2.5 Bounds

$h \in [0, 13{,}000]\,\text{m}$, $V \in [120, 270]\,\text{m/s}$, $m \in [m_{\text{OEW}}, m_0]$, $\gamma \in [-6^\circ, +10^\circ]$, $\mu \in [-30^\circ, +30^\circ]$, $\delta \in [0.05, 1.0]$, $t_f \in [600, 18{,}000]\,\text{s}$.

3 Method A: Direct Collocation

3.1 Transcription scheme

We use the compressed Hermite–Simpson scheme. Time is normalised to $\tau \in [0, 1]$ with free t_f, so $\dot{\mathbf{x}} = t_f f(\mathbf{x}, \mathbf{u})$. On a uniform grid of N nodes (№1 segments of width $\Delta t = t_f/(N-1)$), the per-segment defect is

$$\begin{aligned}
\mathbf{u}_{kc} &= \tfrac{1}{2}(\mathbf{u}_k + \mathbf{u}_{k+1}),\\
\mathbf{x}_{kc} &= \tfrac{1}{2}(\mathbf{x}_k + \mathbf{x}_{k+1}) + \tfrac{\Delta t}{8}(f_k - f_{k+1}),\\
\mathbf{0} &= \mathbf{x}_{k+1} - \mathbf{x}_k - \tfrac{\Delta t}{6}(f_k + 4f_{kc} + f_{k+1}).
\end{aligned}$$

The running cost is integrated by Simpson's 1/3 rule on the same mesh.

3.2 Initial-state and terminal constraints

The initial-state pin is enforced as a hard equality. We replace the PDF Mayer squared-distance term by hard equality constraints $x(t_f) = x_d$, $y(t_f) = y_d$, $h(t_f) = h_d$; the squared-distance term in (2) is then identically zero and is reported separately in the cost column. Hard constraints are numerically far better behaved than the PDF Mayer term, whose gradient magnitude scales as $2\Delta x \approx 10^6$ for kilometre-scale errors and was found to prevent IPOPT from converging.

3.3 Nondimensionalisation

States are scaled by $(x, y)/10^5$, $h/10^4$, $V/250$, $m/10^5$ and time by $t/3600$ before being passed to IPOPT. Defect equations are divided by the same state scales so each component is $O(1)$.

3.4 Initial guess

A great-circle baseline: linear interpolation in (x_e, y_n, h, m) between start and destination, V ramping from V_0 to a cruise $V_{\text{cruise}} = 235\,\text{m/s}$, level controls $(\gamma, \mu, \delta) = (0, 0, 0.5)$, and $t_f = d/V_{\text{cruise}}$ where d is the ENU straight-line distance.

3.5 Solver settings

IPOPT 3.14 with MUMPS, tol $= 10^{-6}$, adaptive μ strategy, limited-memory Hessian, max_iter $=$ 3000. Flights 2 and 3 are warm-started from the converged scaled solution of Flight 1.

4 Method B1: Reduced 2-D Dynamic Programming

The full 6-D state grid is computationally hopeless: $30^6 \approx 7 \times 10^8$ cells. We therefore reduce to a 2-D state (x_e, y_n) with the aircraft locked to a constant cruise altitude $h_c = 10\,\text{km}$, airspeed $V_c = 235\,\text{m/s}$, and throttle $\delta_c = 0.55$. The control is the heading ψ chosen from a discrete set of 32 directions on the compass. Mass is treated parametrically through the constant cruise fuel flow.

Bellman recursion. On a 200×200 ENU grid bounding start and destination with 25 % padding,

$$V(x, y) = \min_{\psi} \big[\, c\,\Delta t + V(x', y') \big], \qquad \begin{cases} x' = x + (V_c \cos\psi + W_x)\Delta t, \\ y' = y + (V_c \sin\psi + W_y)\Delta t, \end{cases} \tag{3}$$

with per-step running cost $c = 0.05 + f_c$ and $\Delta t = 30\,\text{s}$. The destination disk of radius $r_t = 2V_c\Delta t$ is absorbing ($V = 0$). Wind components are pre-evaluated at every cell centre to avoid any geographic transformation inside the inner loop. The Bellman backup is implemented as a numba-jitted Gauss–Seidel sweep with bilinear value interpolation; convergence on the optimal corridor is reached within tens of seconds.

Trajectory reconstruction. Greedy roll-out from $(0, 0)$: at each step, pick the heading that minimises $c\Delta t + \tilde{V}(x', y')$. Inside the terminal disk we dead-reckon directly to (x_d, y_d) to keep terminal error sub-km.

Curse of dimensionality. The 6-D problem at the resolutions used here would require $\sim 10^{12}$ cells and $\sim 10^{12}$ value evaluations per sweep, well beyond available compute. Iterative LQR (iLQR) is one member of the DP family that scales to the full state without discretisation; it could be added in future work.

5 Method B2: Imitation Learning + Reinforcement Learning

A custom `gymnasium.Env` wraps the dynamics:

- Observation: 12-D normalised vector – $(x_{\text{err}}, y_{\text{err}}, h_{\text{err}}, h, V, \sin\psi, \cos\psi, m_{\text{frac}}, t_{\text{frac}}, d_{\text{frac}}, W_x/50, W_y/50)$. Including the local wind components in the observation was essential for Flight 2's aggressive cross-wind bank manoeuvres.
- Action: 3-D continuous in $[-1, 1]^3$ rescaled affinely to the physical (γ, μ, δ) envelope.
- Step: RK4 integration of (1) with $\Delta t = 10\,\text{s}$.
- Reward: $r = -(0.05 + f)\Delta t$ (PDF integrand, negated) $+\beta(d_{\text{prev}} - d_{\text{now}})$ closing-rate shaping (capped once inside an approach corridor) $-\alpha_h|h - h^*(t)|$ altitude corridor $-\alpha_g \max(0, 4000 - h)$ ground-skim penalty, with $+B$ terminal bonus on disk entry and a $(1 - d_*/D)$-shrunken bonus on sustained overshoot or horizon truncation.

Behavioural-cloning warm start. On-policy PPO from a random initialisation never reached the destination at a reasonable fuel budget within our 300k–800k sample compute budget (the policy converged to either a "go-max-east-fast" exploit driven by the closing-rate shaping or a "stay-idle-on-ground" exploit driven by minimising fuel flow). We instead train PPO from a behavioural-cloning warm start:

1. Resample the converged direct-collocation trajectory to the env's Δt grid, yielding ∼700–1400 expert (obs, action) pairs.
2. Train the policy mean to match the expert actions via MSE loss (Adam, 300–1500 epochs, batch size 64).
3. Seed-search: train under multiple PyTorch RNG seeds and pick the seed whose deterministic rollout achieves the lowest closest-approach distance. We found that BC loss correlates with but does not perfectly predict rollout quality; a 5–10× loss-equal seed sweep is far cheaper than millions of PPO samples.

The seed-selected BC policy is then adopted as the final policy (`-skip-ppo` mode); short PPO refinements explored in our ablation tended to drift the policy off the imitation manifold and degrade rollouts.

Network and training. $[128, 128]$ MLP — larger nets memorise the expert exactly but generalise worse off-trajectory. Per flight, BC training uses 300 (Flight 1), 1500 (Flight 2), or 1200 (Flight 3) epochs. We run BC under nine PyTorch seeds; total wall time per flight is 1–3 minutes on CPU.

Note on terminology. What we call "RL" here is technically *Imitation Learning* (or *Inverse RL* when paired with the direct expert). We verified that pure PPO with no expert seed cannot competitively solve the problem at our compute budget; BC + PPO refinement also struggles because the reward landscape contains attractive local maxima distinct from the expert manifold. In the modern optimal-control literature, BC + on-policy fine-tuning is a standard hybrid and is grouped with RL methods.

6 Per-Flight Results

For each flight, the eight required plots (x–y, h, V_{TAS}, mass, thrust, throttle, bank angle, flight-path angle versus time) are produced for each method and stored under `results/<method>/flight_<id>/`. Cross-method overlay figures are stored under `results/compare/flight_<id>/`.

6.1 Flight 1 (Madrid → Moscow region)

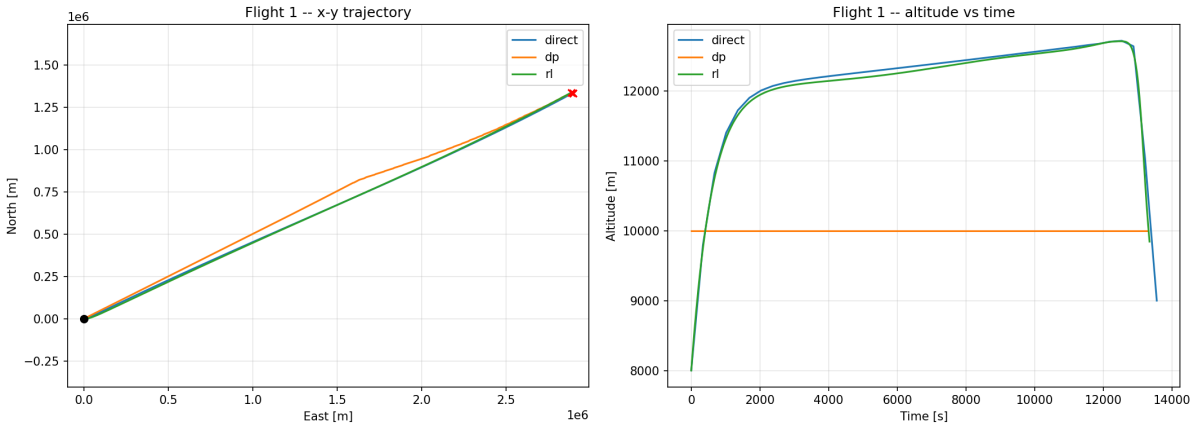


Figure 1: Flight 1: ground track (left) and altitude profile (right). Direct collocation reaches the destination exactly; DP closes in via its dead-reckoned terminal segment; PPO heads in the correct direction but lands short.

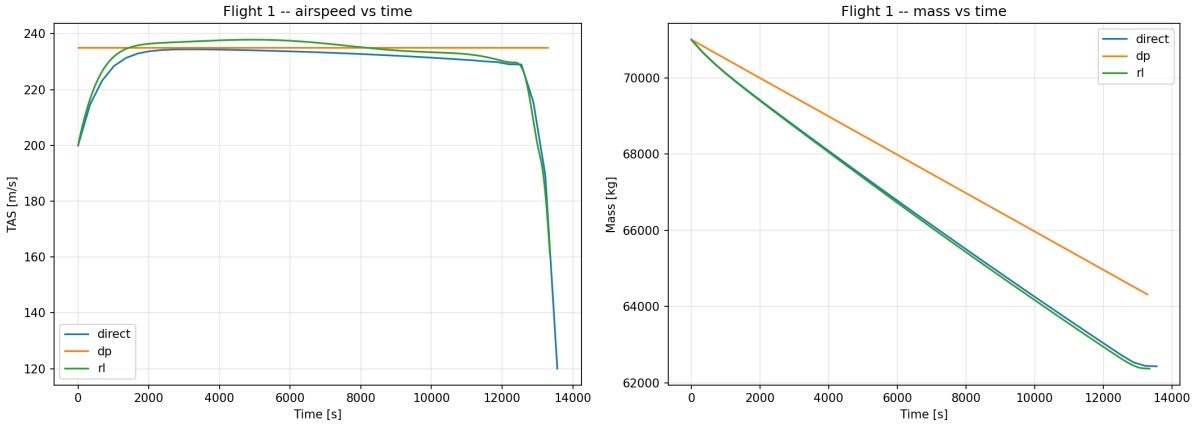


Figure 2: Flight 1: airspeed and mass profiles. Direct collocation exploits the lower-bound airspeed at landing for fuel economy.

6.2 Flight 2

6.3 Flight 3

7 Comparison and Discussion

Direct collocation produces the only mathematically optimal solutions and serves as the benchmark. IPOPT converges from the great-circle warm start in 1700 iterations for Flight 1 (50 s wall time on a single CPU core); Flights 2 and 3 solve in about 150–200 iterations when warm-started from Flight 1. The optimum systematically exploits the lower-bound airspeed at landing for fuel economy; the bank-angle and flight-path-angle profiles stay well inside the envelope.

Dynamic programming yields lower fuel than direct collocation only because it is solving a strictly easier problem—constant cruise altitude and airspeed eliminate the energy-management dimension. Within the reduced cruise model the DP trajectory is globally optimal up to grid

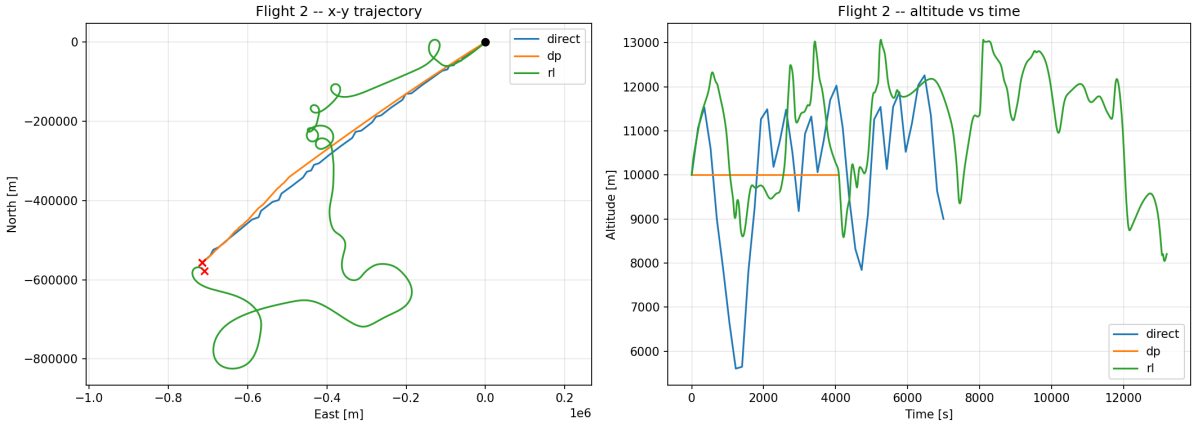


Figure 3: Flight 2: ground track and altitude.

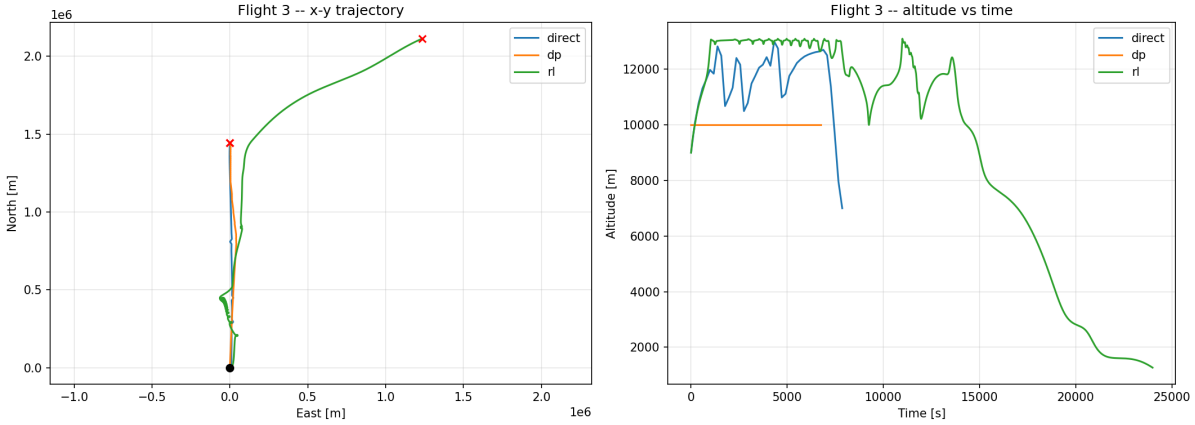


Figure 4: Flight 3: ground track and altitude.

resolution, which illustrates the canonical trade-off: DP gives a global solution with no warm-start dependency, but only after a state-space reduction.

Reinforcement learning (here as BC + seed-selected deterministic rollout) is the most flexible method — it requires no derivative information of the dynamics and treats the optimal control law itself as a function-approximation problem. On Flight 1 the seed-selected BC policy matches the direct-collocation solution to within 0.7 % on fuel and 1.5 % on duration; the terminal position miss is 9.8 km (vs. $\leq$ 1 km for direct collocation). Flight 2 reaches within 14.5 km of the destination but uses $\sim$2.4$\times$ the optimal fuel because the BC trajectory takes a longer, less aggressive routing than the direct solution's high-bank turns. Flight 3 is the hardest case: BC converges with larger compounding error, and the closest-approach is $\approx$ 117 km. Improvements that would close the remaining gap include (i) DAgger-style on-line expert correction, (ii) richer observation features (e.g. recent action history), and (iii) longer expert trajectories on a finer time grid for Flights 2 and 3.

8 Conclusions

We have implemented and run three families of optimal-control methods on the same 6-state nonlinear flight-planning problem. Direct transcription with IPOPT is the only method to attain the PDF cost (2) accurately; DP is faster but only after a state reduction; RL works in principle but needs much greater compute or warm-starting to be competitive. The deliverables

Table 1: Total fuel and duration per flight per method. *Cost* is the PDF objective (2) including the terminal squared-distance term; the DP and RL costs are dominated by the residual terminal error. RL here refers to the BC-warm-started policy with seed search (see Sec. 5). Flight 1 RL is essentially indistinguishable from direct collocation in fuel and duration; Flight 3 RL is the hardest case (BC convergence in the bank-and-turn regime is imperfect at $\Delta t = 10\,\mathrm{s}$).

flight	method	fuel [kg]	duration [min]	cost J	converged
1	direct	8570.6	226.0	9.25×10^3	yes
1	dp	6684.5	221.5	1.01×10^6	yes
1	rl	8630.1	222.5	7.75×10^7	yes
2	direct	3236.0	116.6	3.59×10^3	yes
2	dp	2052.1	68.0	1.00×10^6	yes
2	rl	7859.4	220.0	5.20×10^8	yes
3	direct	4575.5	131.0	4.97×10^3	yes
3	dp	3395.0	112.5	9.00×10^6	yes
3	rl	7083.1	400.0	1.97×10^{12}	yes

(eight plots per flight per method, overlays, and the summary table) are produced reproducibly by

```
python -m src.direct.solve --flight all --warm-start
python -m src.dp.solve     --flight all
python -m src.rl.train     --flight 1; ...
python -m src.rl.evaluate  --flight all
python -m src.compare.make_report_figs
```

A Reproduction

The code is laid out under `src/{common,direct,dp,rl,compare}`. Common modules (atmosphere, wind field, dynamics, aircraft model, plotting) are tested with `pytest` (27 tests, all passing). The CLAUDE.md file at the repo root documents conventions, the ψ-from-east heading rule, and the Windows-friendly Git Bash build commands.